

Final Exam Topics

11. Object oriented programming languages

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Object oriented programming languages

Classes and objects. Encapsulation, members, constructors. Information hiding. Overloading. Memory management, garbage collection. Inheritance, multiple inheritance. Subtyping. Static and dynamic type, type checking. Overriding, dynamic binding. Generics. Subtype and parametric polymorphism. Comparing and copying objects.

1 Class and object

A class is a user-defined type, on the basis of which instances (objects) can be created. A class essentially contains attribute and method (operation) definitions. The class describes the type of the object: it specifies its properties and their possible values (i.e. the type values), as well as the operations that can be performed on the object (type operations).

Example in C++:

```
//definition of Point
class Point {
private:
    int xCoord;
    int yCoord;
public:
    //constructor
    Point(int xCoord, int yCoord) : xCoord(xCoord),yCoord(yCoord) {}

    void Translate(int dx, int dy) {
        xCoord+=dx;
        yCoord+=dy;
    }

    void Translate(Point delta) {
        xCoord+=delta.xCoord;
        yCoord+=delta.yCoord;
    }

    int getX() { return xCoord; }
```

```

    int getY() { return yCoord; }
};

int main(int argc, char* argv[])
{
    Point point(0,0);
    point.Translate(5,-2);
    cout<<point.getX()<<" "<<point.getY()<<endl; //5,-2
    Point delta(-2,1);
    point.Translate(delta);
    cout<<point.getX()<<" "<<point.getY()<<endl; //3,-1

    return 0;
}

```

Remark: Non-object-oriented languages do not have classes, e.g. older languages such as C, ADA, Pascal, or functional languages (Haskell, Clean, etc.).

2 Encapsulation

Encapsulation expresses the idea that related data and functions/procedures are together, belonging to a single unit. Another important concept is data hiding, which means that from the outside only what the object's class permits is directly accessible. This is important in order to prevent unwanted dependencies from forming, to make the code easier to understand, and to avoid the uncontrolled spread of data (see object orgy/feature envy).

If an object or class hides all of its data members, and clients can only access them through certain methods, then encapsulation realizes a strong form of abstraction and information hiding. Some languages, such as Java or C++, C#, even enforce this (public: accessible to everyone, private: only to objects of the given class, protected: only to instances of the given class or its descendant classes), while others, such as Python, do not – there only conventions can achieve something similar (please don't directly poke at things whose name starts with an underscore). Java and C# also have package-level visibility, which is the default in Java. These characteristics can be assigned both to data members and to methods.

Data hiding supports refactoring, that is, the internal representation of the class can be rewritten more freely without affecting clients, as long as they can still call the existing public methods with the same parameterization. Furthermore, it encourages programmers to put related data and the functions/procedures that process them in one place, an organization that the programmer's peers can also understand. It also makes loosening of dependencies (coupling) easier.

3 Constructor, destructor

3.1 Constructor

The constructor is the initializing procedure of objects; it runs when a new object is instantiated from a class. By default constructor we mean the parameterless constructor, which in most programming languages is automatically generated by the compiler with an empty body if no constructor at all was specified in a class. Among other things, it is customary in the constructor to take care of allocating storage for the object's variables with dynamic lifetime.

3.2 Destructor

The destructor is the opposite counterpart of the constructor; this procedure runs when objects are released. Its call happens automatically, regardless of whether the release of the object happens automatically at the end of

the scope (for local objects), manually by the programmer (for objects with dynamic lifetime), or by the garbage collector (also for objects with dynamic lifetime).

In the destructor it is customary, among other things, to release the object's dynamically allocated data members and the acquired resources.

4 Information hiding

The object hides its data and certain operations. This means that we do not know exactly how the data is represented inside an object, and indeed we do not even know the implementations of the operations. Hiding information serves the safety of the object, which we can only access through controlled operations.

5 Overloading and overriding

5.1 Overloading

Several functions with the same name but different signatures. The actual parameters of the function call determine which function will be invoked. This is already decided at compile time (static, compile-time binding).

With the help of overloading we can create subprograms with the same name but different signatures. In most programming languages the signature means the name of the subprogram together with the number and types of the formal parameters, but in some languages (for example Ada) the type of the return value also belongs to it. The primary use of overloading is to be able to perform the same activity with different parameterizations.

From the subprogram call, the compiler can decide which of the overloaded variants should be invoked. If none matches or several match, a compile error occurs.

5.2 Overriding

Within a class hierarchy, a descendant class redefines a method of the ancestor class (same name, same signature). If we access instances of the class hierarchy through a pointer or reference of the ancestor type and through it call the overridden method, then it is decided at run time exactly which method will be invoked (dynamic, run-time binding).

6 Copying and comparing objects

Copying objects is done with a special constructor, the so-called copy constructor. It receives an instance of the given class as a parameter, and copies it into the object currently being initialized according to the operating logic specified by the programmer.

The compilers of several programming languages (for example C++) automatically create a copy constructor if the programmer does not define their own. This default copy constructor copies the values of all data members of the source object; this is called a shallow copy. If the object also contains dynamically allocated data members, then it is not their value but only their reference that gets copied, which is usually not the desired behavior. In this case writing a custom copy constructor is necessary, one that makes a deep copy.

7 Automatic, static and dynamic lifetime, garbage collection

Lifetime: by the lifetime of variables we mean the section of the program's execution time during which the storage allocated for the variable belongs to the variable.

7.1 Automatic lifetime

Local variables declared inside blocks have automatic lifetime, which means it lasts from the declaration to the end of the containing block, i.e. it coincides with the scope. Storage for them is allocated in the current activation record of the execution stack.

7.2 Static lifetime

Global variables, and in some languages variables declared as static (for example in C/C++ with the `static` keyword), have static lifetime. The lifetime of such variables extends over the entire execution time of the program; storage for them can already be allocated at compile time.

7.3 Dynamic lifetime

For variables with dynamic lifetime the programmer allocates storage for them on the dynamic storage area (heap), and it is also the programmer's task to ensure that this storage area is later released. If they forget about the latter, this is called a memory leak. As we can see, in the case of dynamic lifetime the scope is not connected to the lifetime in any way; the lifetime can be narrower or wider than the scope.

7.4 Garbage collector

The other name of the garbage collector is the waste collector; after the English name Garbage Collector it is often abbreviated to just GC. Its task is to automate the storage release associated with dynamic memory management, and to take the responsibility off the programmer's shoulders, thereby reducing the chance of errors.

The garbage collector observes which variables have gone out of their scope, and declares them releasable. The disadvantages of the method are its computational cost and its nondeterminism. The garbage collector does not release variables that have gone out of scope immediately, and the order of release is not the same as the order in which the variables became releasable.

When and which variable the garbage collector releases is determined by a complex algorithm that is specific to each programming language, in which the saturation of the available memory and the estimated time needed for release usually play a role. (For example, if an object has a destructor, then the GC usually releases it only later.)

Overall, the garbage collector only guarantees that sooner or later (at the latest by the end of the program's run) it releases every dynamically allocated variable.

Languages that use garbage collection are e.g. Java, C#, Ada. In C/C++ there is no garbage collection; the programmer has to take care of releasing the dynamically allocated memory areas.

8 Inheritance

A class can most simply be created by listing its data members and methods. However, the object-oriented paradigm also offers another, more efficient method: inheritance. With reusability in mind, inheritance makes it possible to create a new (child, descendant, sub-) class starting from an already existing (parent, ancestor, super-) class. Inheritance is a relationship between two classes in which the descendant class has almost all the properties of the parent class (it treats its non-private data members and methods as its own), and can supplement these with new ones. The class created this way can itself be the ancestor of other classes (except, for example, in Java classes marked with the `final` keyword, from which we can no longer derive), so these classes are organized into an inheritance hierarchy. Depending on whether a class has one or more ancestors, we speak of single or multiple inheritance. Java supports single inheritance, but for example C++ has multiple inheritance.

In Java the topmost element of the class hierarchy is the `Object` class, from which every other class (directly or indirectly) derives.

A subclass can reimplement the inherited methods. In this case the given method is realized with the same name but a different, modified (subclass-specialized) content. Such methods are called polymorphic. In Java every method that is not marked with the `final` keyword can be redefined. (In C++ everything that is not private.)

Examples:

- C++:

```
class RegularPolygon
{
protected:
    const double radius;
public:
    RegularPolygon(double radius) : radius(radius) {}
    virtual double area() = 0;
};

class EquilateralTriangle : public RegularPolygon
{
public:
    EquilateralTriangle(double radius) : RegularPolygon(radius) {}
    virtual double area()
    {
        return 0.75*sqrt(3.0)*radius*radius;
    }
};
```

- Java:

```
public abstract class RegularPolygon{
    protected final double radius;

    public RegularPolygon(double radius){
        this.radius = radius;
    }
    public abstract double area();
}

public class EquilateralTriangle extends RegularPolygon{
    public EquilateralTriangle(double radius){
        super(radius);
    }
    public double area(){
        return 0.75*Math.sqrt(3.0)*radius*radius;
    }
}
```

9 Static and dynamic type and binding

It is not the client's task but the object's task to choose how to react to a method call. It typically does this at run time, and selects the method call from its associated table. This is known as dynamic binding, and it distinguishes the object from the abstract data type and the module, which have a fixed implementation for every instance. If the other parameters also have a say in selecting the method, then we speak of multiple binding (see double method, multimethod, multiple method).

The method call is also regarded as message passing, where the client sends a message to the object participating in the binding.

- static type: The type specified in the variable's declaration. It is unambiguously decided during compilation and cannot change during run time. The static type determines what is allowed to be done with the object (e.g. which operations can be called on it).
- dynamic type: The type of the object referenced by the variable. It is either a descendant of the static type, or the static type itself. It can change during run time.

Example (Java):

```
Object o = new String("Hello");
```

Here the static type of variable `o` is `Object`, while its dynamic type is `String`.

Bindings:

- static binding: One can refer to the data members according to the variable's static type.
- dynamic binding: The data members according to the variable's dynamic type can be used.

Binding is important when the called operation, or the referenced variable, differs between the static and the dynamic type, or possibly does not even exist in the static type (in which case we cannot refer to the data member in the dynamic type either).

Bindings can be determined in several ways in different languages:

- Java : There is dynamic binding in every case (including in the bodies of inherited methods).
- C++ : When defining the operation, one can indicate if dynamic binding is wanted (`virtual`).
- Ada : At the call one can indicate if dynamic binding is wanted.

10 Generics

Generic programming: Programming algorithms and data structures in a generalized way that works for several types (e.g. generic sort, generic stack, ...). This general code is called a template.

In certain languages (Ada, C++) a template must be instantiated – at this point the desired types and objects (according to the template definition) are given to the template as parameters and the template in question gets instantiated. (Just like, for example, when we give a function its actual parameter.) This is called generative programming: based on the given template the program creates a “real” program unit (a program generates a program).

Example: When instantiating a stack template we specify what type of elements the stack should store (type parameter) and how many elements fit into the stack (object parameter). In the case of C++ and Ada a template parameter can also be a subprogram.

Example: We tell a sort by what operation it should sort. Naturally there can also be a default template parameter.

When defining a template, naturally the template cannot be used for every type. A template becomes a template by working for several types. However, we can (and must) place restrictions on which types it can be used with and how the template can be instantiated.

A good example of this is the Ada language, where the specification of the template is a “contract” between the body of the template and the instantiation:

- The body of the template cannot use anything other than what the template’s specification permits it. (The body does not necessarily have to be known at instantiation time.)

```
generic
  type Element_T is private;
  with function "*" (X, Y: Element_T) return Element_T is <>;
function Square (X : Element_T) return Element_T;
```

Here the `is <>` at the end of the line starting with `with function` means that if a `*` operation already exists for the given type (e.g. for integers), then it does not have to be specified separately at instantiation; the program automatically uses it.

The body:

```
function Square (X: Element_T) return Element_T is
begin
  return X * X;  -- The formal operator "*".
end Square;
```

- The instantiation must provide everything that the template’s specification requires of it. In the following example we apply the squaring function to matrices, assuming we have defined matrix multiplication.

```
with Square;
with Matrices;
procedure Matrix_Example is
  function Square_Matrix is new Square
    (Element_T => Matrices.Matrix_T, "*" => Matrices.Product);
  A : Matrices.Matrix_T := Matrices.Identity;
begin
  A := Square_Matrix (A);
end Matrix_Example;
```

For example in C++ the template contract does not work this way. There the template’s specification is the entire definition (because of this, template classes can only be defined entirely in header files). At instantiation this too must be known, so that we know how we can instantiate. The principle of information hiding is thus violated.

In other languages (e.g. Java, functional languages) templates do not have to be instantiated – always the same written code is executed, just with the current parameters. E.g. in Java the type parameter is “lost” at compile time, it only turns out at run time.

11 Polymorphism

In programming languages and type theory, polymorphism refers to a unified interface that is implemented by different types. It also means many-formedness. Operations performed on a polymorphic type can be applied to values of several different types. Polymorphism can be realized in several ways:

- **Ad-hoc polymorphism:** A function has many different implementations, each specified individually for some different types and their combinations. It can be realized with overloading.
- **Parametric polymorphism:** The code is written generally for different types, and can be applied to all types that satisfy certain conditions predefined in the code. In an object-oriented context it is called a template or generic. In a functional programming context it is simply called polymorphism.
- **Subtyping:** The name denotes instances of several different classes that have a common ancestor declaring the function.[3] In an object-oriented context this is mostly what people mean when they speak of polymorphism.

11.1 Parametric polymorphism

The characteristic of parametric polymorphism is that the function has to be written once and it will work uniformly for every type. The types of the function's parameters have to be specified as parameters when the function is used, hence the name parametric. It is often also called generic, because of the uniform behavior. Its use increases the expressive power of the language while preserving full type safety.

Parametric polymorphism can be interpreted not only for functions but also for data types, so data types can also be parametric, in other words generic. Generic types can also be found under the name polymorphic data types.

Parametric polymorphism is very common in functional programming, which is why it is often simply called polymorphism. The following Haskell example presents a parametrically polymorphic list data type, with two parametrically polymorphic functions.

```
class List<T> {
    class Node<T> {
        T elem;
        Node<T> next;
    }
    Node<T> head;
    int length() { ... }
}

List<B> map(Func<A, B> f, List<A> xs) {
    ...
}
```

In object-oriented languages it is less common, but most of them also provide the possibility of parametric polymorphism. This is provided by templates in C++ and D, and by generics in Java.

11.2 Subtyping

In some languages subtyping can be used to constrain polymorphism. In these languages subtyping makes it possible for a function to use an object of a certain type T, but if S is a subtype of T, then the function can also be used for objects of type S, according to the Liskov substitution principle. The subtype is sometimes denoted $S <: T$. Conversely, T is a supertype of S, denoted $T >: S$. Subtype polymorphism is usually dynamic.

In the following example dog and cat are subtypes of animal. The `letsHear()` procedure accepts an animal, but also works with subtype objects.


```
abstract class Animal {
    abstract String talk();
}

class Cat extends Animal {
    String talk() {
        return "Meow!";
    }
}

class Dog extends Animal {
    String talk() {
        return "Woof!";
    }
}

void letsHear(final Animal a) {
    println(a.talk());
}

int main() {
    letsHear(new Cat());
    letsHear(new Dog());
}
```

In the following example Number, Rational, and Integer are types such that $\text{Number} \rightarrow \text{Rational}$ and $\text{Number} \rightarrow \text{Integer}$. A function whose parameter is Number works with an Integer or Rational parameter just as with a Number parameter. The actual type can be hidden from the user, and accessed by object identity. In fact, if the Number type is abstract, then there cannot even be objects whose actual type is Number. See: abstract class, abstract data type. This type hierarchy is also known as the numeric tower from the Scheme programming language, and usually consists of even more types.

Object-oriented languages realize subtype polymorphism with inheritance. In typical implementations classes contain a virtual table, which references the functions implementing the polymorphic part of the class's interface. Every object contains a reference to this table, which is always needed when something calls a polymorphic function. This mechanism is an example of:

- Late binding, because virtual function calls are linked only at the call;
- Single dispatch, since virtual function calls take into account only the virtual table of their first argument, so the dynamic type of the other arguments is not considered.

Some object systems, for example the Common Lisp Object System, are also capable of multiple dispatch, so calls become polymorphic in all arguments.